Figma → 代码：组件复用验证报告

痛点 4「Figma 到实现存在偏差，组件未被充分复用」· 仓库 luckylucky0001/figma-fe · 2026-10-20 · 交付物：CLI `figmafe`（Go）+ Skill `figma-to-code` + 免费 Figma 导出插件 + Demo + 分步教程 + 演示 deck

26 个真实开源项目（H5 / Android 主流技术栈）× 3 份设计稿	**827/858** 项复用推荐正确，**0** 项错误，其余为「可用但不是最优」	**0/11 → 11/12** 真实项目里实现新页面：不用 Skill → 用 Skill 通过复用检查	**+35%** token、+45% 耗时：用 Skill 多出的成本

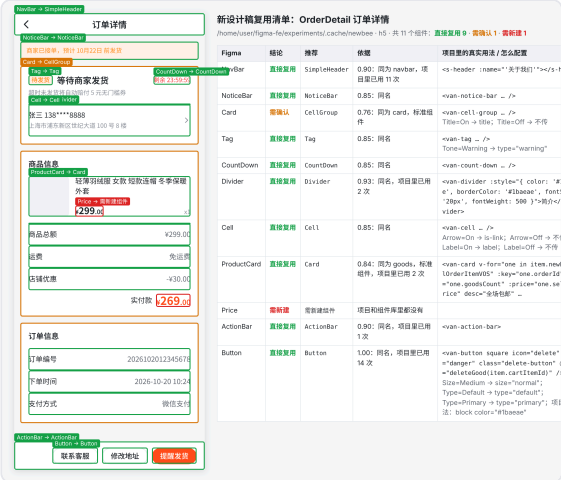
结论

- **对存量项目，先把「有什么」查清楚，才知道新设计稿能复用什么。** `figmafe reuse` 扫一遍现有 H5（Vue / React / uni-app / Taro + Vant / antd-mobile / NutUI / uni-ui / taro-ui）或 Android（XML View / Compose，QMUI / Material / AppCompat / 自有设计系统）项目，得到组件资产、项目惯例、重复实现和「组件库有但本项目用不了」的组件。`figmafe match` 再把新设计稿逐个元素对上去，给出「直接复用 / 需确认 / 需新建」，并附上项目里的真实写法和变体 → props 的配置。
- **准确率**：26 个项目 × 3 份设计稿共 858 项，✓827 / △31 / ×0。其中 21 个是「留出」项目：调规则时没用过，第一次跑出现过错误（见第 2 节），修的都是通用规则，之后作为回归测试。包括芋道商城、gkd、AndroidProject 三个大型系统（第 3 节）。
- **真实实现**：在 vue-h5-template 里让 Agent 按 Skill 实现新页面，复用门禁、类型检查、构建都通过；像素对比因项目主题和设计稿不同没通过，Agent 反馈的工具问题已修（第 4 节）。
- **Agent 实测**：在 5 个真实项目里按同一份设计稿实现新页面，不用 Skill 的 11 次全部没通过复用检查（Now in Android 里新建了 11～12 个和已有组件重复的组件；Haiku 在 newbee 里一个现成组件都没用），用 Skill 的 11/12 次通过。Haiku 从 0/4 提到 4/4：模型越弱，固定流程的收益越大。
- **新页面闭环**：demo 仓库里，`figmafe verify` 一条命令输出复用率、违规、像素差异和节点级偏差，退出码可做 CI 门禁；故意埋的 3 处错误全部被抓出，修正后一次通过。
- **只用免费能力**：不用 Figma Code Connect、Dev Mode MCP 和 Variables REST API（都要付费），设计数据走自己写的插件（Plugin API，免费）。

1. 方案：Proposal 的每一条都落成了命令

① 体检存量	② 新稿复用清单	③ 人确认	④ 实现后检查	⑤ 对照设计	⑥ 不许变差
`reuse`：资产、惯例、重复概念、复制粘贴结构、写死颜色、版本 / 主题兼容性	`match`：Figma 元素 → 项目组件，附真实用法、props 配置、可视化叠图	需确认 / 需新建的项写进 `component-map.yaml`，下次自动采用	`reuse --focus` / `--changed`：自造轮子、有组件没用、手写已有组件	`compare` / `verify`：截图 + 像素 / 节点级偏差，带 Figma 节点 id	`--baseline --no-regress` 接 CI；偏差沉淀成 `rules.md`

Proposal 要求	落地方式
Figma 组件与代码组件的对应关系	`match` 按名字、UI 概念（中英文同义词）、使用次数、设计系统封装打分；人确认后写进 `component-map.yaml`，之后 match 优先采用确认过的映射
开发前识别可复用组件	先 `reuse` 再 `match`：只推荐**本项目真能用**的组件（检查组件库版本、改名 / 移除、主题依赖、废弃 API、设计系统是否封装过）；没有组件但有固定写法时推荐「项目惯例」
开发后检查用了哪些组件、和 Figma 截图对比	`reuse --focus <页面>` 或 `--changed <git ref>` 只看新增代码；`compare` 截图对比；demo 里 `verify` 把两者合成一个门禁
偏差反馈给 Agent、沉淀规则	每条问题都带文件行号或 Figma 节点 id 和修改建议，Skill 规定照着改到通过；决定和理由写进 component-map，经验写进 `rules.md`



newbee-mall（Vue 3 + Vant）上跑「订单详情」的复用清单：绿框直接复用、橙框需确认、红框需新建；右侧是项目里的真实写法和 Figma 变体对应的 props。

2. 复用清单准不准：26 个真实开源项目（固定 commit，可复现）

项目	订单详情	我的持仓	买入下单
newbee	✓11	✓12	✓10
qmui	✓11	✓12	✓8 △2
nia	✓11	✓12	✓9 △1
antdm-template	✓11	✓12	✓10
antdm-mobile	✓11	✓12	✓10
jetsnack 留出	✓10 △1	✓11 △1	✓10
litemall 留出	✓11	✓12	✓10
hello-uniapp 留出	✓11	✓12	✓9 △1
taro-ui-demo 留出	✓11	✓12	✓10
nutui-vite 留出	✓11	✓12	✓10
sunflower 留出	✓10 △1	✓11 △1	✓10
pokedex 留出	✓11	✓12	✓10
newpipe 留出	✓10 △1	✓11 △1	✓8 △2

项目	订单详情	我的持仓	买入下单
vant-demo-vue3 留出	✓11	✓12	✓10
vant-demo-vue2 留出	✓11	✓12	✓10
vue3-vant-mobile 留出	✓11	✓12	✓10
mall-app-web 留出	✓10 △1	✓12	✓10
antennapod 留出	✓10 △1	✓11 △1	✓9 △1
wanandroid 留出	✓11	✓12	✓10
seal 留出	✓10 △1	✓11 △1	✓9 △1
kunminx 留出	✓10 △1	✓11 △1	✓7 △3
jeecg-uniapp 留出	✓11	✓12	✓10
vue-h5-template 留出	✓10 △1	✓12	✓10
yudao-mall 留出	✓10 △1	✓11 △1	✓10
gkd 留出	✓10 △1	✓11 △1	✓10
androidproject 留出	✓10 △1	✓11 △1	✓10

合计 ✓827 / △31 / ×0（858 项）。技术栈：Vue 2/3、React、uni-app、Taro × Vant / NutUI / antd-mobile / uni-ui / taro-ui / wot-design-uni / 自有组件；Android XML / Compose × QMUI / Material / AppCompat / 三方 View 库 / 自有设计系统（逐项见 `experiments/real-match-eval.md`）。订单详情是组件化设计稿，我的持仓未组件化、中文图层，买入下单未组件化、含新概念。

✓ 推荐正确；△ 可用但不是最优（多为「需确认」）；× 推荐错误。留出 = 调规则时没用过的项目。

留出项目第一次跑的结果（如实记录，修正前）

项目	第一次	出错原因 → 通用修正
Jetsnack	订单详情 3 ×	把桌面小部件（Glance）组件当成 App 组件；没认出 `Jetsnack*` 是项目前缀 → 识别 Glance、学习项目前缀、`ui/components` 算设计系统
litemall	两份各 1 ×	推荐了 Vant 3 才有的 ActionBar，项目是 Vant 2 → 组件库加入改名 / 移除信息，按实际安装版本过滤
hello-uniapp	持仓页 12 ×	uni-app 内置小写标签被当成项目组件、中文图层名规范化后为空被判「同名」→ 加 uni-ui 目录，修空名误判
NewPipe	3 ×	AppCompat 标准控件没算进项目资产，XML 页面被推荐 Compose 组件 → 跨技术栈的候选最多「需确认」
mall-app-web / AntennaPod（第二批）	崩溃 + 1 × / 1 ×	uni-app 内置标签没有 import 路径导致崩溃；`<view>` 用量压低了 uni-ui 的占比；AntennaPod 的空状态封装 EmptyViewHandler 不是 View → 修崩溃、占比排除平台内置元素、识别「界面封装类」
WanAndroid / Seal / KunMinX / jeecg（第三批）	11 / 7 / 3 / 11 ×	三方 View 库和 wot-design-uni 没被收录；Android 的 ActionBar 是顶栏；「Video*」被当成项目前缀；PasswordInputView / ConfirmButton 这类专门用途组件被当成通用组件；会崩溃的 Material 组件没有别的候选时仍排第一 → 逐条改成通用规则
vue-h5-template（第四批）	23 ×	同时装了 Vant / NutUI / Varlet，打分打平后一半推荐 NutUI，同一页面混用两套组件库；演示页被当成设计系统 → 先定「主用组件库」（看 `.env` 里的 UI 配置项），目录按最外层分类（见第 4 节）
芋道商城 / gkd / AndroidProject（第四批，大型系统）	崩溃 / 5 × / 8 ×	props 三元默认值让扫描崩溃；图标按钮被当成文字按钮；54 个 AppCompatButton 漏数；「验证码倒计时」被当成倒计时组件（逐项见第 3 节）
买入下单（新设计稿）	14 个项目 12 ×	开关 / 复选框 / 单选原来是一个概念，搜索算在输入框里 → 拆开概念、补词表、检查 Material 3 主题

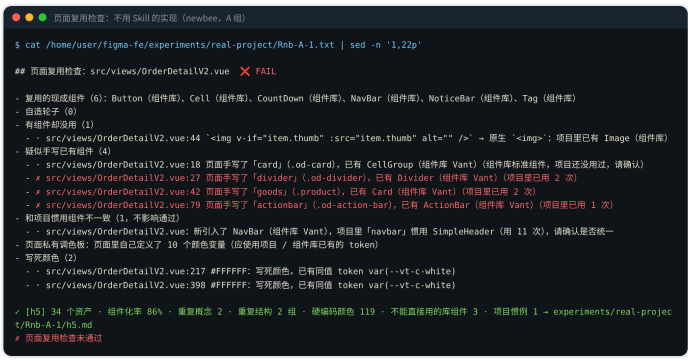
vant-demo、Pokedex、Sunflower、vue3-vant-mobile 第一次跑就没有 ×。复用推荐只错在「项目没见过的结构」，修正都是通用规则，没有针对单个项目的特判；`python3 experiments/real-match-eval.py` 每次改规则都会全量回归。

体检：组件库有，不等于本项目能用

写完检查：有现成组件却手写



QMUI_Android 用的是 AppCompat 主题和 Material 1.4：这些组件会崩溃或编译失败，match 不会推荐。



不用 Skill 的实现（newbee，A 组）：分割线、商品卡、底部操作栏都是手写的，每条带行号和应复用的组件。

3. 三个大型系统实测（H5 + Android，用 Skill 和 CLI 分析）

前面的项目多是模板和示例。为了看工具在规模大、写法杂的生产系统上表现如何，又挑了三个国内常见的大型开源系统：H5 商城芋道（uni-app）、Compose 应用 gkd、国内很多 App 的骨架 AndroidProject（XML）。每个系统都按 Skill 的顺序跑：`figmafe reuse` 体检 → `figmafe match` 出复用清单 → 人工逐条判断 → 出错的改成通用规则 → 26 个项目全量回归。标准答案在跑当前版本之前写好；「第一次跑」是这一轮修正前的版本（commit 19762fb）。

系统	订单详情	我的持仓	买入下单	第一次跑
芋道商城 yudao-mall-uniapp（H5 · uni-app）	✓10 △1	✓11 △1	✓10	扫描崩溃
gkd（Android · Compose Multiplatform）	✓10 △1	✓11 △1	✓10	5 ×
AndroidProject（Android · XML View）	✓10 △1	✓11 △1	✓10	8 ×

芋道商城 yudao-mall-uniapp（H5 · uni-app） commit 4924ad51

uni-app 商城：自有设计系统 `sheep/ui`（su-*）+ 业务组件 `sheep/components`（s-*）+ uni-ui



复用清单叠图（订单详情（组件化设计稿））：绿框直接复用、橙框需确认、红框需新建。

307 个文件	122 个组件资产	99% 组件化率	11 个重复概念

组件库用量：uni-app 内置组件 374、uni-ui 128；设计系统封装：Swiper → SuSwiper

体检优先处理

1. 628 处写死的颜色已有同值 token，可以直接替换（见第 6 节）
2. 「button」有 2 种实现（共用 187 次）→ 统一到 `Button`
3. 「goods」有 3 种实现（共用 50 次）→ 统一到 `SGoodsColumn`

复用清单（订单详情（组件化设计稿）：直接复用 9 · 需确认 1 · 需新建 1）

设计稿元素	结论	推荐	依据
NavBar	直接复用	`SuNavbar`	同名，项目里已用 1 次，设计系统组件
NoticeBar	直接复用	`SuNoticeBar`	同名，变体轴匹配 1/1，项目里已用 1 次，设计系统组件
Card	直接复用	`Card`	同名
Tag	直接复用	`Tag`	同名，项目里已用 1 次
CountDown	直接复用	`SCountDown`	同名，项目里已用 1 次
Divider	直接复用	`SuSubline`	同为 divider，项目里已用 1 次，设计系统组件
Cell	直接复用	`ListItem`	同为 cell，标准组件，项目里已用 6 次
ProductCard	直接复用	`SGoodsColumn`	同为 goods，项目里已用 34 次
Price	需新建	—	项目和组件库里都没有
ActionBar	需确认	`GoodsNav`	同为 actionbar，标准组件
Button	直接复用	`Button`	同名，项目里已用 186 次

第一次跑（这一轮修正前）：**扫描直接崩溃**（三份设计稿都跑不出来）。**现在**：订单详情 ✓10 △1 · 我的持仓 ✓11 △1 · 买入下单 ✓10。

- Vue props 里 `a ? b : c` 形式的默认值让 props 解析越界 → 冒号前必须有键名（加回归测试）
- su-image 被当成 `<image>` 的封装，但项目直接写 `<image>` 71 次、su-image 4 次 → 封装组件用得不到原生一半时不算封装
- 同一颜色有多个 token 时随手挑了一个 → 选项目里引用最多的 token
- 卡片推荐了 SLiveCard（直播卡片）、分割线推荐了装修用的 SLineBlock → 商城领域词（live / coupon / groupon / wallet …）算专门用途；分割线别名加 subline（真正的分割线是 su-subline）
- 体检把装修组件 s-notice-block（只是包了一层 su-notice-bar）算成第二种实现 → 记录组件模板里用到的组件，建立在同概念组件之上的算特化

还剩的问题：底部操作栏：项目的 `detail-tabbar.vue`（4 个商品页在用）按钮从插槽传入，文件本身看不出是操作栏，工具给的是 uni-ui GoodsNav（需确认）

gkd（Android • Compose Multiplatform） commit b2b42f28

Jetpack Compose + Material3，自有 `ui/component`（GkTopAppBar、GkSettingItem、GkSwitch …）



复用清单叠图（买入下单（未组件化、英文图层））：绿框直接复用、橙框需确认、红框需新建。

839 个文件	175 个组件资产	100% 组件化率	4 个重复概念

组件库用量：Compose Material3 174、Compose Foundation 20；设计系统封装：Checkbox → GkCheckbox、FloatingActionButton → GkAnimatedFloatingActionButton、ModalBottomSheet → GkModalBottomSheet、Switch → GkSwitch、TextField → GkMultiTextField、TopAppBar → GkTopAppBar

体检优先处理

1. 「dialog（Compose）」有 3 种实现（共用 36 次）→ 统一到 `GkDialog`
2. 「card（Compose）」有 4 种实现（共用 30 次）→ 统一到 `Card`
3. 「input（Compose）」有 3 种实现（共用 18 次）→ 统一到 `GkMultiTextField`

复用清单（买入下单（未组件化、英文图层）：直接复用 8 • 需确认 1 • 需新建 1）

设计稿元素	结论	推荐	依据
NavBar	直接复用	`GkTopAppBar`	同为 navbar，项目里已用 28 次，设计系统组件
SearchBar	直接复用	`GkAppBarTextField`	同为 search，6 处调用里有 6 处传的是搜索词，是项目的搜索框，项目里已用 6 次，设计系统组件
Tabs	需确认	`TabRow`	同为 tabs
Cell	直接复用	`GkSettingItem`	同为 cell，项目里已用 23 次，设计系统组件
Stepper	需新建	—	项目和组件库里都没有
Input	直接复用	`GkMultiTextField`	同为 input，项目里已用 6 次，设计系统组件
Switch	直接复用	`GkSwitch`	同名，可选中组件，设计稿里是静态展示，项目里已用 7 次，设计系统组件
Checkbox	直接复用	`GkCheckbox`	同名，可选中组件，设计稿里是静态展示，项目里已用 6 次，设计系统组件
Button	直接复用	`TextButton`	同为 button，项目里已用 72 次
Empty	直接复用	`GkEmptyState`	名称包含，项目里已用 15 次，设计系统组件

第一次跑（这一轮修正前）：订单详情 9 ✓ / 1 △ / 1 ×；我的持仓 8 ✓ / 1 △ / 3 ×；买入下单 8 ✓ / 1 △ / 1 ×。**现在**：订单详情 ✓10 △1 • 我的持仓 ✓11 △1 • 买入下单 ✓10。

- 根目录只装了 husky 的 `package.json` 让它被当成 H5 项目，自动识别平台时全部「需新建」→ 依赖里有前端框架才算 H5
- 按钮推荐了 GkIconButton（图标按钮 71 处），文字按钮其实是 TextButton（72 处）→ icon 算专门用途词
- 搜索框是放在顶栏里的 GkAppBarTextField，名字里没有 search → 调用处大多绑定搜索词（`value = searchStr`）的输入组件按用法认作搜索框
- 体检「button 有 11 种实现」「dialog 有 15 种实现」「GkTwoLineText 是分割线」→ 专门用途 / 弱词组件不算重复；按 Composable 函数体记录它建立在哪个组件之上（重复概念 9 → 4）
- RadioButton 被判成「已被 GkMenuItemRadioButton 封装」→ 封装组件 = 库组件名前加前缀和一个词，且不是业务命名

还剩的问题：涨跌标签推荐了 M3 InputChip（需确认）：项目里没有标签组件，严格说应新建

AndroidProject（Android · XML View） commit cdf35d36

XML + DataBinding 骨架：TitleBar 三方库 + 自有 `customWidget` 库（SettingBar、SwitchButton、RegexEditText …）+ StatusLayout



复用清单叠图（买入下单（未组件化、英文图层））：绿框直接复用、橙框需确认、红框需新建。

198 个文件	41 个组件资产	100% 组件化率	0 个重复概念

组件库用量：Android SDK widgets 103、com.hjq.bar 13、com.hjq.shape 5、Material Components 1、com.airbnb.lottie 1、com.hjq.nested 1、me.relex.circleindicator 1

体检优先处理

1. 5 处写死的颜色已有同值 token，可以直接替换（见第 6 节）
2. 23 个元素的结构被复制了 2 次（app/src/main/res/layout/dialog_activity.xml:2 等）→ 抽成组件
3. 1 处原生 Toolbar：项目里已有 TitleBar（组件库）

复用清单（买入下单（未组件化、英文图层）：直接复用 7 · 需确认 1 · 需新建 2）

设计稿元素	结论	推荐	依据
NavBar	直接复用	`TitleBar`	同为 navbar，标准组件，项目里已用 13 次
SearchBar	需新建	—	项目和组件库里都没有
Tabs	需确认	`TabLayout`	同为 tabs，项目几乎不用 Material Components，标准组件
Cell	直接复用	`SettingBar`	同为 cell，项目里已用 14 次
Stepper	需新建	—	项目和组件库里都没有
Input	直接复用	`RegexEditText`	同为 input，项目里已用 14 次
Switch	直接复用	`SwitchButton`	同为 switch，项目里已用 8 次
Checkbox	直接复用	`CheckBox`	同名，系统原生控件，项目里已用 3 次
Button	直接复用	`Button`	同名，系统原生控件，项目里已用 54 次
Empty	直接复用	`StatusLayout`	同为 empty，项目里已用 4 次

第一次跑（这一轮修正前）：订单详情 8✓ / 1△ / 2×；我的持仓 8✓ / 1△ / 3×；买入下单 7✓ / 3×。**现在**：订单详情 ✓10 △1 · 我的持仓 ✓11 △1 · 买入下单 ✓10。

- 布局里 54 个 `AppCompatButton` 没算成平台 Button，于是推荐了只用过 1 次的 SubmitButton → AppCompat* 计入对应的平台控件
- 设置行 SettingBar、状态布局 StatusLayout 没被认出 → 概念词补上 settingbar / settingitem / statuslayout
- CountdownView 的注释写着「验证码倒计时」，是发送验证码的按钮 → 读组件声明前的文档注释，写明专门用途的降权，结论最多「需确认」
- 体检把 FloatActionButton / PlayButton 算成按钮的重复实现 → 专门用途组件不算重复（重复概念 1 → 0）

还剩的问题：支付倒计时其实要新建：工具给的是「需确认 CountdownView（专门用途：验证码）」，由人来否掉

4. 真实实现：Agent 按 Skill 在 vue-h5-template 里做订单详情页

sunniejs/vue-h5-template 是国内常用的 Vue 3 + Vant H5 模板（commit 9ac24512）。这是唯一一个真的安装依赖、跑类型检查、构建和像素对比的项目：让 Agent 按 Skill 实现「订单详情」，再用同一套命令验收。

① 体检
装了 Vant、NutUI、Varlet 三套组件库，`.env` 里 `VITE_UI_FRAMEWORK=vant` → 主用 Vant

② 复用清单
第一次 10 ✓ / 23 × （一半推荐 NutUI，照做会混用）→ 修正后 32 ✓ / 1 △ / 0 ×

③ 实现
新页面 `src/views/order/detail.vue` + 新建 `PriceText.vue`，component-map 写明理由

④ 验收
复用门禁通过 · `vue-tsc` 通过 · `vite build` 通过 · 50 个单测通过



存量体检：sunniejs/vue-h5-template（figmafe reuse）

存量复用体检：/tmp/claude-0/cn/vue-h5-template（h5）

技术栈：vue / Vant / NutUI · 扫描文件 69

指标	值
组件化率（组件使用 / 组件使用 + 可替换的原生元素）	33%（41 / 125）
重复概念（同一 UI 概念有多个实现）	5
重复结构（可抽成组件的复制粘贴结构）	0 组
硬编码颜色	4 处（其中 1 处已有同值 token）

0. 优先处理（按影响面排序）

1. 51 处原生 `<button>`：项目里已有 Button（组件库 Vant）
2. 18 处原生 `<input>`：项目里已有 Field（组件库 Vant）
3. 「icon」有 3 种实现（共用 16 次）→ 统一到 `SvgIcon`
4. 8 处原生 `<select>`：项目里已有 Field（组件库 Vant）
5. 「button」有 3 种实现（共用 7 次）→ 统一到 `Button`

1. 三方组件库使用

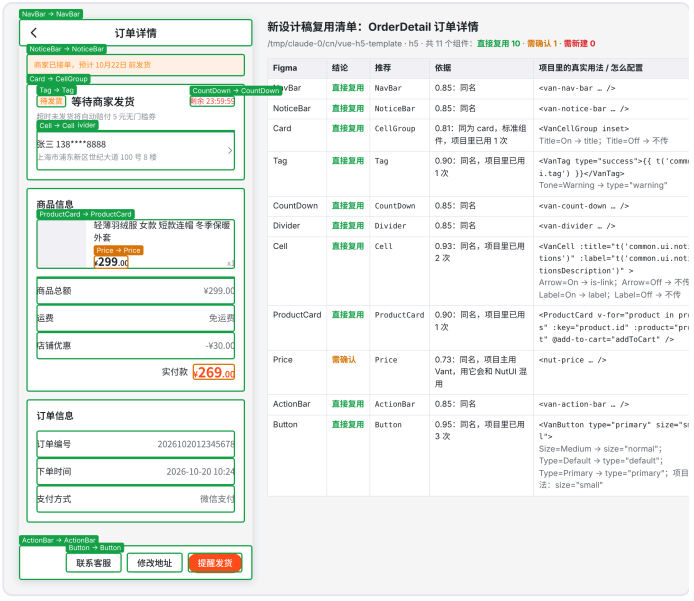
组件库	使用次数	最常用
NutUI	9	Button×3、Cell×2、CellGroup×1、Progress×1、Switch×1、Tag×1
Vant	9	Button×3、Cell×2、CellGroup×1、Progress×1、Switch×1、Tag×1

主用组件库：Vant（依据：.env 里 VITE_UI_FRAMEWORK=vant）。装了多个组件库时，新页面只用它，不要和其他组件库混用。

组件库里有、但本项目不能直接用的组件（match 不会推荐它们）：

- `GoodsActionButton`：Vant 3.0.0 起已改名为 ActionBarButton
- `GoodsActionIcon`：Vant 3.0.0 起已改名为 ActionBarIcon

体检报告：主用组件库 Vant（依据 .env），列出另外两套组件库的用量。



新设计稿复用清单：OrderDetail 订单详情

/tmp/claude-0/cn/vue-h5-template · h5 · 共 11 个组件：直接复用 10 · 需确认 1 · 需新建 0

Figma	结论	推荐	依据	项目里的真实用法 / 怎么配置
NavBar	直接复用	NavBar	0.85：同名	`<van-nav-bar … />`
NoticeBar	直接复用	NoticeBar	0.85：同名	`<van-notice-bar … />`
Card	直接复用	CellGroup	0.81：同为 card，标准组件，项目里已用 1 次	`<VanCellGroup inset>` Title=On → title；Title=Off → 不传
Tag	直接复用	Tag	0.90：同名，项目里已用 1 次	`<VanTag type="success">{{ t('comm i.tag') }}</VanTag>` Tone=Warning → type="warning"
CountDown	直接复用	CountDown	0.85：同名	`<van-count-down … />`
Divider	直接复用	Divider	0.85：同名	`<van-divider … />`
Cell	直接复用	Cell	0.93：同名，项目里已用 2 次	`<VanCell :title="t('common.ui.not tions')" :label="t('common.ui.not tionsDescription')" >` Arrow=On → is-link；Arrow=Off → 不传 Label=On → label；Label=Off → 不传
ProductCard	直接复用	ProductCard	0.90：同名，项目里已用 1 次	`<ProductCard v-for="product in pr s" :key="product.id" :product="pr t" @add-to-cart="addToCart" />`
Price	需确认	Price	0.73：同名，项目主用 Vant，用它会和 NutUI 混用	`<nut-price … />`
ActionBar	直接复用	ActionBar	0.85：同名	`<van-action-bar … />`
Button	直接复用	Button	0.95：同名，项目里已用 3 次	`<VanButton type="primary" size="s l">` Size=Medium → size="normal"；Type=Default → type="default"；Type=Primary → type="primary"；项目法：size="small"

修正后的复用清单叠图：全部推荐 Vant，Price 只在 NutUI 里有，标「需确认」。

像素对比 7.11%，没通过。差异来自项目主题：靛蓝主色、44px 高的按钮、字体都和设计稿不同。组件复用对了，样式还要对齐项目的主题 token：这正是存量项目的常态，新页面应该用项目的主题，而不是照抄设计稿的颜色。

Agent 反馈的工具问题	处理
ProductCard 推荐了商城网格里的 ProductCard，实际用了 Vant Card	两种都合理，回归用例的标准答案收下两者
Size=Medium 按名字映射成 `size="normal"`（44px），设计稿按钮 32px	**已修**：按设计稿里实例的高度选尺寸，给出「Size=Medium（设计稿高 32px）→ size="small"（Vant small = 32px），按名字会对到 normal（44px）」
布局组件 Layout（含 `<router-view>` 和固定底栏）被当成操作栏候选	**已修**：含 router-view 的组件是页面外壳，不算任何栏
Card 映射到 CellGroup 的 title，但 title 显示在 inset 卡片外面	**已修**：属性提示里写明「标题要在卡片里时，用第一行 Cell 当标题」

5. Agent 实测：真实项目里实现一个新页面

同一份设计稿（订单详情），在 5 个开源项目里新建页面。A 组只给任务和 Figma 导出，C 组加上 Skill（体检 → 复用清单 → 实现 → 页面检查）。所有 run 用同一版 `figmafe reuse --focus / --changed` 打分：自造已有组件、有组件没用、手写已有组件（按 class / id 或横排「标题 + 值」结构识别）任何一项都算不通过。

项目	技术栈	A 不用 Skill（Opus / Haiku）			C 用 Skill（Opus / Haiku）		
		通过	复用组件数	问题数	通过	复用组件数	问题数
newbee-mall	Vue 3 + Vant	0/3	6 / 6 / 0	4 / 4 / 14	3/3	12 / 12 / 10	0 / 0 / 0
litemall（留出）	Vue 2 + Vant 2	0/1	1	14	2/2	10 / 9	0 / 0
QMUI_Android	XML + QMUI	0/3	8 / 9 / 6	1 / 1 / 1	3/3	9 / 8 / 8	0 / 0 / 0
NewPipe（留出）	XML + AppCompat	0/1	3	2	1/1	6	0
Now in Android	Compose + 自有设计系统	0/3	2 / 2 / 1	6 / 6 / 2	2/3	7 / 8 / 8	2 / 0 / 0
合计		0/11	**平均 4.0**	**共 55**	11/12	**平均 8.9**	**共 2**

- **Now in Android**：A 组没先查已有组件，在 core/designsystem 下新建了 NiaNavBar / NiaCard / NiaDivider / NiaCell 等，和已有的 NiaTopAppBar / Card / HorizontalDivider / ListItem 重复，还写了一套 18 个写死颜色的主题。C 组复用了它们，新建的组件都在 component-map.yaml 里写了理由。
- **QMUI / NewPipe**：C 组发现 match 推荐了项目用不了的组件（MaterialDivider 需要 Material 1.5、项目是 1.4；Chip 在 AppCompat 主题下会崩）。这两类问题已经做进工具：体检会列出「组件库有、但本项目不能直接用的组件」，match 不再推荐。实验中 agent 报告的工具问题都已修复并加了回归测试。
- **成本**：有用量记录的 4 次 A、10 次 C，C 平均多 35% token、45% 时间，多在体检、复用清单、页面检查和写决定这几步。

6. 新页面实现闭环（demo 仓库）

- 自建的 H5（React）和 Android（Compose，兼容 compileSdk 32 / minSdk 23；另有 XML View 存量组件）组件库加一份订单详情设计稿，跑了 13 次 A / C 对照。`verify` 能区分「页面写错」和「组件本身不符合设计」（见 `docs/component-issues.md`）；教程第 8 步故意写错 3 处，全部被抓出。
- 这个仓库小、组件好找，A 组复用率也是 100%，**证明不了复用率提升**；差别在规范：A 组 7/8 次自造了设计稿外的 CountDown，C 组 0/5。复用率的差距要在真实项目里看（第 3 节）。

7. 边测边改：准确率随项目变化

阶段	项目数	结果	这一阶段新增的通用规则（举例）
报告初版	22	✓699 △27 ×0	版本 / 主题兼容性检查、项目惯例、设计系统封装
+ vue-h5-template	23	✓734 △25 ×0	装了多套组件库时定主用库；只有依赖里的包才算三方库
+ 三个大型系统	26	✓826 △32 ×0	读组件文档注释；AppCompat* 算平台控件；建立在其他组件之上的算特化
继续迭代（当前）	26	✓827 △31 ×0	按设计稿高度选尺寸；按调用处用法认出搜索框

每加一批项目，先按人工写好的答案跑一次、如实记录错误，再修成通用规则；每次修改都重跑 26 个项目和真实项目实验的打分（`experiments/rescore-real.sh`），这几轮打分都没有变化。

8. 局限（如实说明）

- **没有真实 Figma 文件**，本环境也访问不了 Figma：三份设计稿由手写的设计源生成（格式和插件导出一致）。插件只用 mock 的 Plugin API 测过，没在真实 Figma 里跑过。
- **大多没有编译**：除了 vue-h5-template（第 4 节），Agent 实验只做静态检查，没有构建和运行这些开源项目（本环境装不了 Android SDK）；demo 的 Android 截图用 Compose Desktop 渲染，不是真机。
- **实验有泄漏**：A 组 agent 也能看到本仓库的 Skill；1 次 A run 自己跑了 figmafe，已剔除。泄漏只会让 A 组更好，所以 A / C 差距是保守估计。样本量小（每格 1～2 次），只能说明方向。
- **评分标准是人定的**：✓ / △ / × 由我逐条判断并写成回归用例；真实接入时应由组件 owner 复核一次。

- **只看静态代码**：用途只能从插槽或运行时看出的组件（芋道的 detail-tabbar）工具认不出；组件库的尺寸表目前只收录了 Vant / NutUI 的按钮。

9. 接入你们的项目

#	动作	产出
1	`figmafe reuse <H5 / Android 仓库>`（只读，不改代码）	资产清单、优先处理项、首份体检基线
2	Figma 插件导出下一个要做的页面，`figmafe match`	复用清单 + 叠图，组件 owner 确认「需确认」项
3	开发时让 Agent 按 Skill 走；提交前 `reuse --changed origin/main`	新增代码的复用问题，带行号
4	CI 里 `reuse --baseline main.json --no-regress`（示例：`docs/ci/`）	组件化率、写死颜色等指标只许变好

教程：`docs/tutorial/README.md` · 数据：`experiments/real-match-eval.md`、`match-judgement.md`、`real-project/summary.md`